

NUCLEO-G474RE

Power Electronics Project Series

Project 06

Buck Converter Closed Loop Co-Simulation

Hardware-in-the-Loop co-simulation of a closed-loop buck converter using
STM32G474RE and MATLAB/Simulink

Implementation Framework:
MATLAB Embedded Coder & Simulink

Document Version: 1.0

Last Updated: June 11, 2026

GitHub: <https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide>

This work is licensed under the MIT License

Contents

1	Project Overview	3
1.1	Objective	3
1.2	Applications	3
1.3	Key Concepts	4
1.4	Learning Outcomes	4
2	MATLAB Embedded Coder Implementation	4
2.1	Prerequisites	4
2.2	MATLAB/Simulink Model Design	5
2.2.1	Model 1 — Power Converter (PC-Side Plant Simulation)	5
2.2.2	Model 2 — Controller (Deployed on STM32)	6
2.3	Peripheral Configuration	8
2.3.1	Pin Configuration	8
2.3.2	ADC Configuration	8
2.3.3	DAC Configuration	9
2.3.4	Timer Configuration	9
2.3.5	LPUART1 Configuration	9
2.3.6	USART3 Configuration	10
2.4	Wiring and Hardware Setup	11
2.5	Building and Deploying	12
3	Testing and Validation	12
3.1	Hardware Testing	12
3.2	Model Simulation Testing	12
3.3	Integration Testing	12
4	Results and Analysis	13
4.1	Experimental Results	13
4.2	Key Observations	14
5	Troubleshooting	15
6	Extensions and Enhancements	15
6.1	Possible Improvements	15
6.2	Known Limitations	15
7	Conclusion	16
A	Controller: buck_controller Function	18
B	Controller: parse_rx Function	20
C	Converter: parse_uart Function	21

D Hardware Documentation

21

1 Project Overview

1.1 Objective

This project implements a hardware-in-the-loop (HIL) co-simulation of a closed-loop buck converter using the STM32G474RE microcontroller in conjunction with MATLAB/Simulink R2025a. Unlike the dual-framework approach used in earlier projects, this project is **MATLAB-only**, as the co-simulation workflow is inherently Simulink-based.

The architecture is deliberately split across two independent Simulink models to work around the memory constraints of the STM32G474RE. Deploying both the controller and a high-fidelity Simscape converter model simultaneously on the microcontroller would exceed its available Flash and RAM. The two-file method resolves this: the **controller model** is deployed as a standalone executable on the STM32, whilst the **plant (converter) model** runs purely in simulation on the PC.

A key differentiator from conventional co-simulation approaches is the use of the onboard ADC and DAC to physically close the control loop. The STM32 receives the simulated output voltage from the PC via USART3, outputs it through the DAC pin (PA4), and this DAC output is shorted externally to the ADC input pin (PA0) using a jumper wire. This arrangement simulates actual analogue sampling rather than bypassing the ADC entirely as most conventional UART-based co-simulations do.

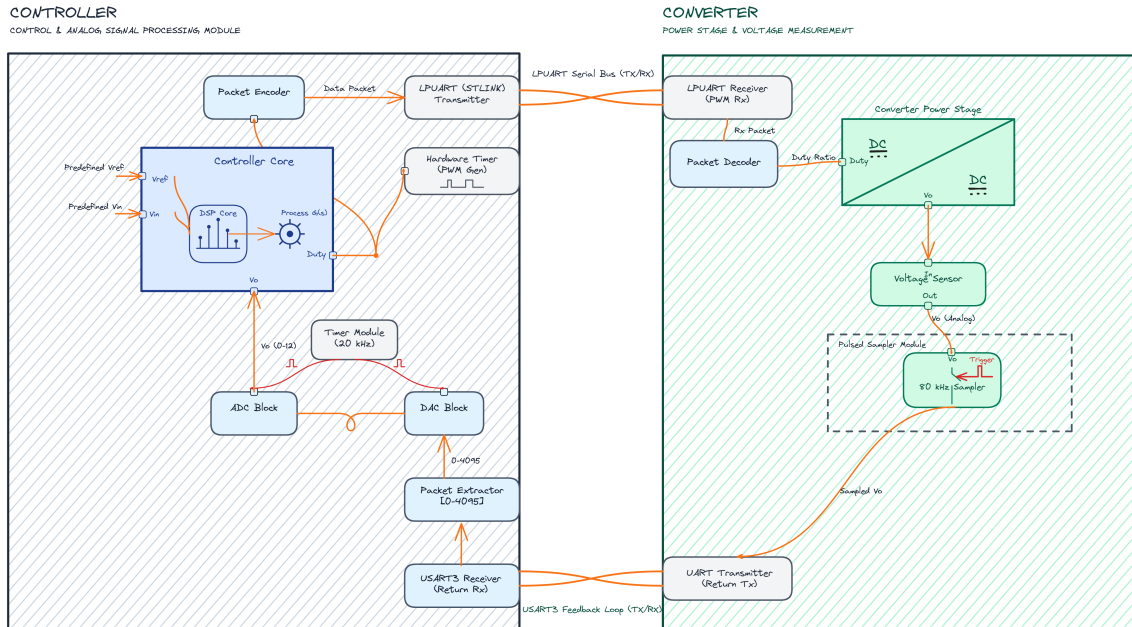


Figure 1: System architecture: controller deployed on STM32(Blue), plant simulated on PC(Green).

1.2 Applications

- Rapid prototyping and validation of closed-loop power converter control algorithms without dedicated HIL hardware
- Educational demonstration of digital control implementation on embedded targets

- Feasibility testing of PI and other control laws against high-fidelity Simscape plant models prior to hardware deployment

1.3 Key Concepts

- **Power Electronics:** Buck converter topology, switching frequency, duty cycle, and voltage regulation
- **Digital Control Systems:** PI control, feedforward compensation, anti-windup, hysteresis, and discrete-time implementation on an embedded target
- **Hardware-in-the-Loop (HIL) Co-simulation:** Splitting plant and controller across physical and simulated domains; closing the loop via analogue I/O

1.4 Learning Outcomes

By completing this project, you will:

- Understand how to partition a control system across an STM32 target and a PC-based Simscape simulation
- Appreciate the role of the ADC and DAC in faithfully sampling and reconstructing signals within a co-simulation loop
- Gain practical experience configuring DMA, UART, TIM, ADC, and DAC peripherals for a real-time control application
- Understand the design trade-offs in a tightly constrained embedded control loop

2 MATLAB Embedded Coder Implementation

2.1 Prerequisites

Before starting this implementation, ensure you have:

- *MATLAB R2025a with Embedded Coder and Simscape toolboxes installed*
- *Embedded Coder Support Package for STMicroelectronics STM32 (install via Add-On Explorer)*
- *ARM GCC Compiler configured in MATLAB — change from default to **GNU Tools for ARM Embedded Processors***
- *NUCLEO-G474RE development board*
- *USB-to-USART converter (FTDI chip recommended; must support at least 2 Mbaud and be 3.3 V logic compatible)*
- *Familiarity with Simulink block diagrams and basic power electronics*

Note: The ARM GCC compiler configuration is critical for successful code generation. By default, MATLAB may be configured with a different toolchain. This must be changed to **GNU Tools for ARM Embedded Processors** for STM32 compatibility.

Note: It is strongly recommended to complete the previous projects in this series before attempting this one, as they establish the foundational workflow for Simulink deployment on the STM32. The full project series is available at the project repository.

2.2 MATLAB/Simulink Model Design

This project comprises two separate Simulink models that communicate with each other in real time via serial (UART) links.

2.2.1 Model 1 — Power Converter (PC-Side Plant Simulation)

The converter model runs entirely on the PC in normal Simulink simulation mode. It receives the duty cycle from the STM32, computes the theoretical output voltage using a Simscape buck converter, and transmits that voltage back to the STM32.

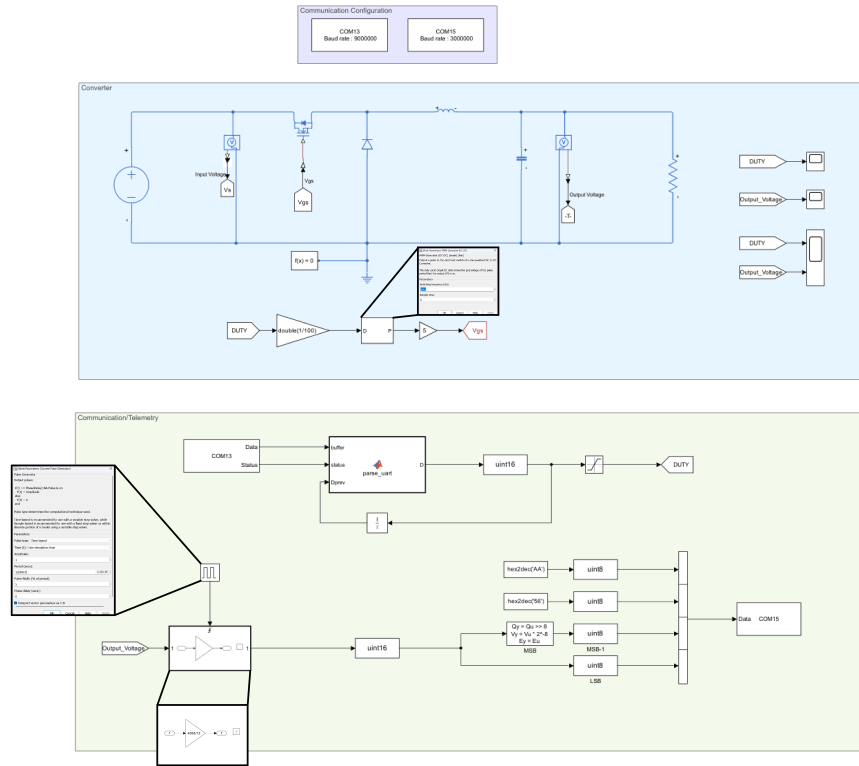


Figure 2: Power converter Simulink model: Simscape plant (blue), communication configuration (purple), and data transmission subsystem (green).

The model is organised into three functional areas:

- **Simscape Buck Converter (blue):** The core power converter plant model. The duty cycle arrives as a percentage from the STM32; it is first divided by 100 to bring it within the 0–1 range required by the PWM Generator (DC-DC) block. The switching frequency is set to 20 kHz.

- **Communication Configuration (purple):** Configures the serial communication ports and baud rates for both UART links used in the co-simulation.
- **Data Transmission (green):** Handles bidirectional data flow.
 - *Receive path:* COM13 receives the duty cycle from the STM32. A `parse_uart` function (see Appendix C) reconstructs the duty ratio from the incoming packet.
 - *Transmit path:* The output voltage is measured via a Simscape voltage sensor and scaled into the DAC input range of 0–4095. This conversion is performed on the PC to avoid burdening the STM32 with computation unrelated to the control law. The gain for this scaling resides within a triggered subsystem, sampled at 80 kHz with a 5% duty ratio. The 80 kHz sampling rate satisfies the Nyquist criterion with respect to the 20 kHz control law ($f_s > 2f_m$). The value is cast to `uint16` and split into MSB and LSB bytes. These are concatenated with a **packet header** (which allows the receiver to detect the start of a new packet) and a **stream ID** (useful for extending to multiple signals). The final four-byte packet is then transmitted to the STM32.

2.2.2 Model 2 — Controller (Deployed on STM32)

The controller model is compiled and deployed directly onto the STM32G474RE as a standalone embedded executable.

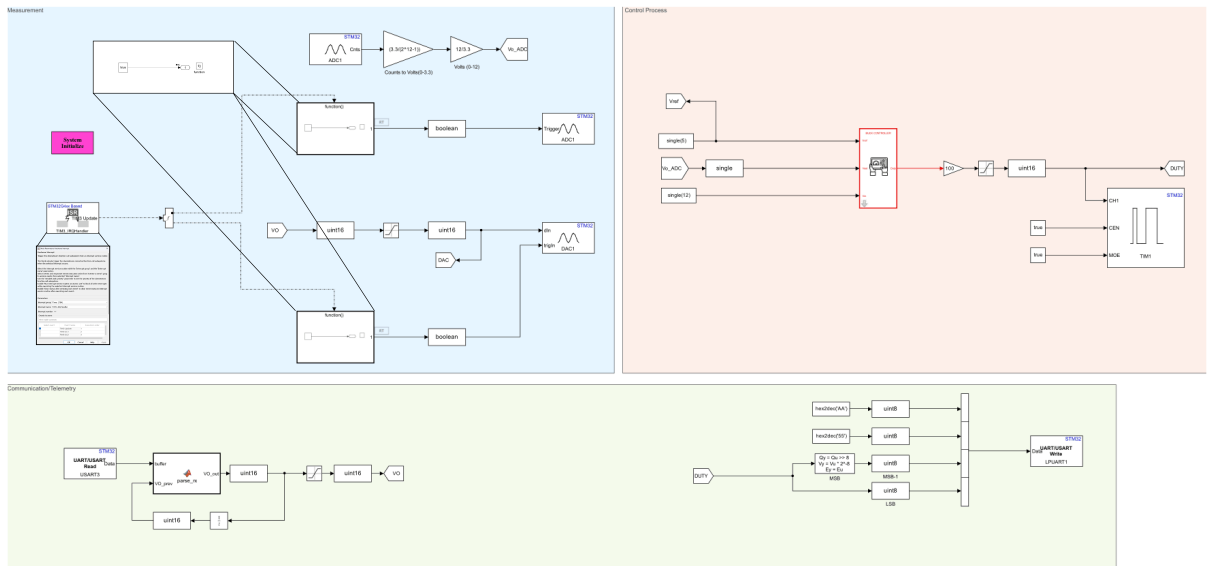


Figure 3: Controller Simulink model: measurement and ADC/DAC processing (blue), PI control law (red), and telemetry/communication (green).

The model is divided into three functional areas:

Measurement (Blue — Top Left) TIM3 is configured to generate update events at 20 kHz. An ISR block detects these events, and a **Function Call Split** block routes the single ISR trigger simultaneously to two function call subsystems — one to trigger the ADC read and another to trigger the DAC write. This ensures both peripherals are synchronised to the same 20 kHz timebase without requiring separate timers.

Once the ADC samples the voltage present on PA0 (which is physically connected to the DAC output PA4 via a jumper wire), the raw ADC count is passed through a gain block that maps the 0–4095 range to 0–12 V. This gives V_o , the estimated output voltage. A buck converter operating from a 12 V rail cannot produce a sustained output exceeding 12 V (excluding brief transients), so this range is physically justified.

Control Process (Red — Top Right) The control subsystem takes three inputs:

- V_{ref} — the desired output voltage setpoint (fixed constant)
- V_{in} — the input supply voltage (fixed constant)
- V_o — the measured output voltage from the ADC path

These feed into the `buck_controller` MATLAB Function block (see Appendix A), which implements a feedforward PI controller with gain scheduling, asymmetric hysteresis, and anti-windup. The output is a duty ratio (0–1), which is multiplied by 100 to express it as a percentage before transmission. Saturation blocks are present throughout to clamp signals within safe bounds. The duty signal also feeds into the TIM1 PWM module to produce physical PWM pulses at 20 kHz for waveform verification.

Telemetry / Communication (Green — Bottom)

- *Transmit (LPUART1)*: The duty cycle (as a percentage, cast to `uint16`) is packed into a four-byte structure — header, stream ID, MSB, LSB — and transmitted to the PC-side converter model via LPUART1.
- *Receive (USART3)*: The simulated V_o value (as a 0–4095 DAC integer) arrives via USART3. A `parse_rx` function (see Appendix B) extracts the `uint16` value from the packet. This value is written to the DAC (PA4), which physically drives the voltage on PA4. Since PA4 is shorted to PA0, the ADC on the next ISR trigger reads back this voltage, completing the closed loop.

Note: DMA is used wherever possible throughout the controller model — for ADC, DAC, LPUART1, and USART3 — to offload data transfers from the processor core. This keeps the CPU free to execute the control law deterministically at 20 kHz. Additionally, the DAC output buffer is disabled, which enables near-instantaneous voltage transitions on PA4 — beneficial for this sampling arrangement.

2.3 Peripheral Configuration

All peripheral configuration is done in STM32CubeMX, accessed through the MATLAB hardware setup workflow. The `.ioc` file is generated and opened in CubeMX to configure each peripheral as described below.

2.3.1 Pin Configuration

Table 1: Pin Configuration for Project 06

Pin	Peripheral	Description
PA0	ADC1 CH1	Analogue input — shorted to DAC output (PA4) via jumper
PA4	DAC1 OUT1	DAC output — drives simulated V_o voltage onto PA0
PA2	LPUART1 TX	Transmits duty cycle (%) to PC converter model
PA3	LPUART1 RX	Not actively used in this flow
PB10	USART3 TX	Not actively used in this flow
PB11	USART3 RX	Receives simulated V_o from PC converter model
PA8	TIM1 CH1	Physical PWM output at 20 kHz for waveform verification

Note: LPUART1 (PA2/PA3) is routed through the onboard Micro-USB connector (ST-LINK VCP). USART3 (PB10/PB11) exposes only the TX and RX pins directly; an external USB-to-USART converter is required. An FTDI-based converter is recommended — verify it supports at least 2 Mbaud and is **3.3 V logic compatible**. If the converter has a voltage selection jumper, ensure it is set to 3.3 V before connecting.

2.3.2 ADC Configuration

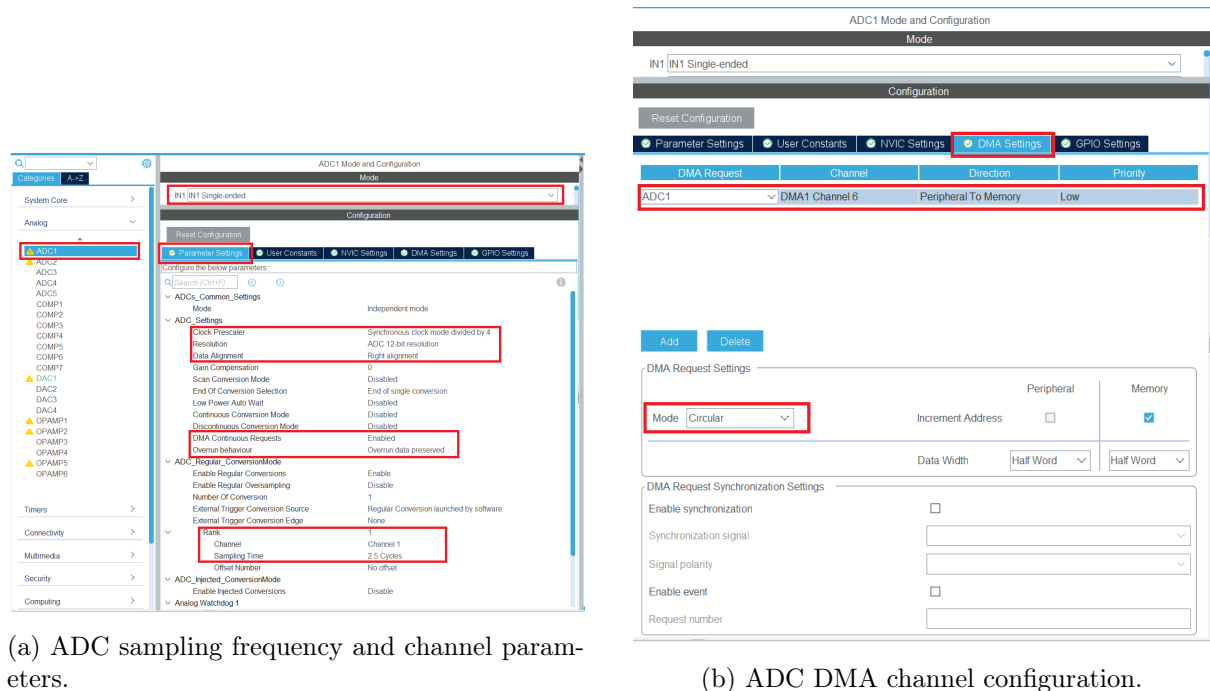


Figure 4: ADC1 peripheral configuration in CubeMX.

2.3.3 DAC Configuration

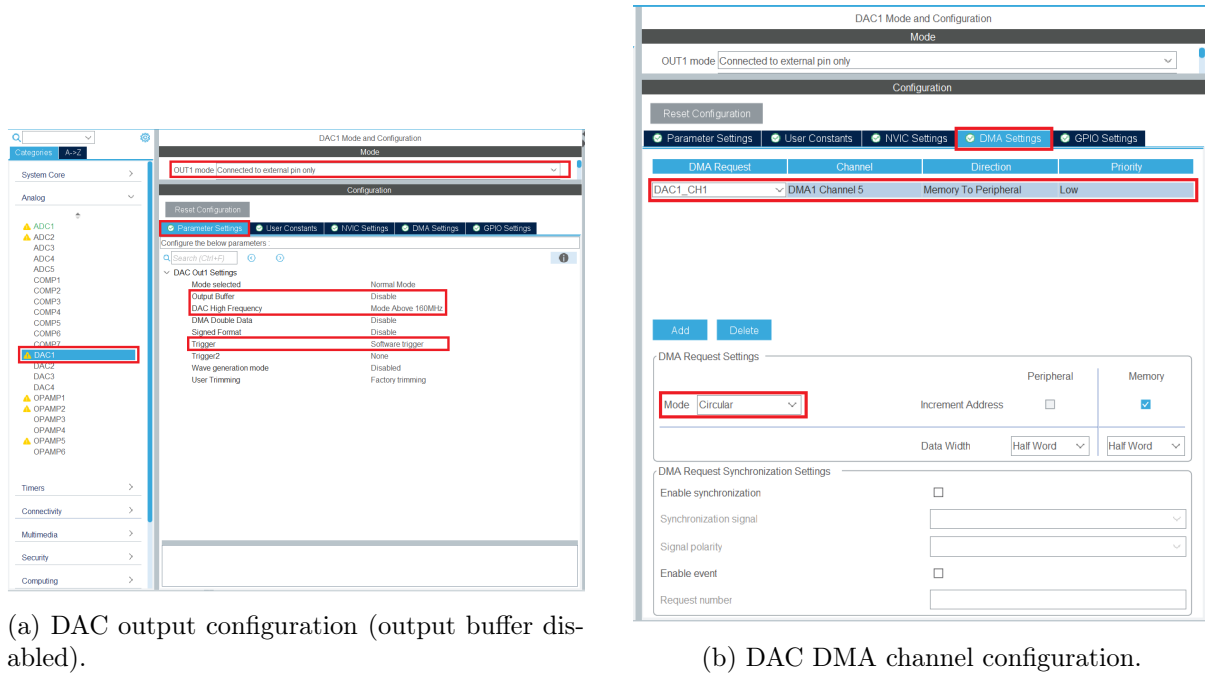


Figure 5: DAC1 peripheral configuration in CubeMX.

2.3.4 Timer Configuration

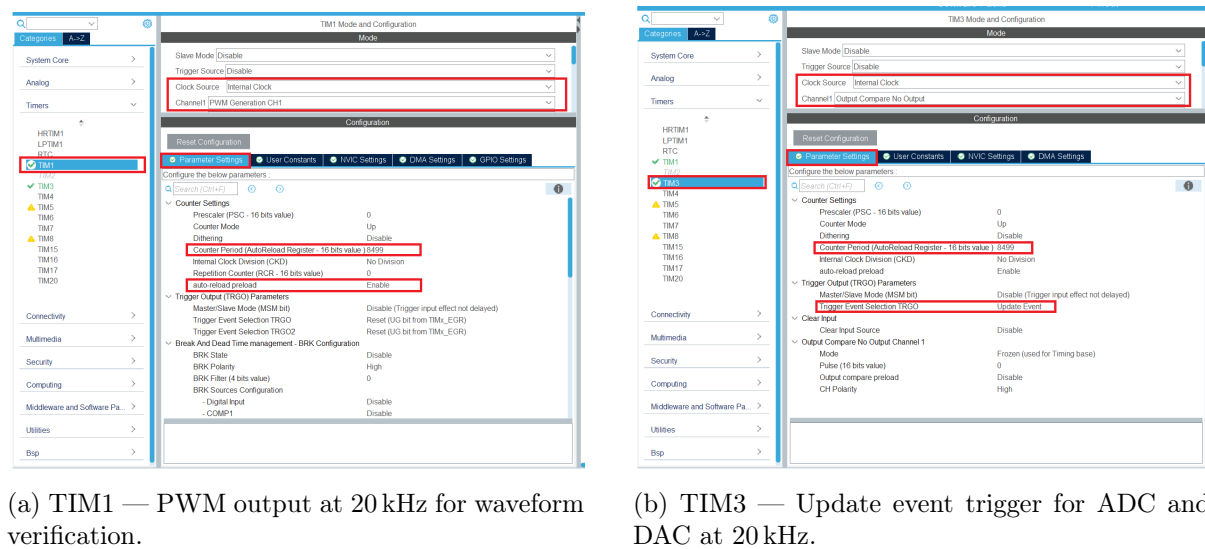


Figure 6: Timer peripheral configuration in CubeMX.

2.3.5 LPUART1 Configuration

LPUART1 handles transmission of the duty cycle from the STM32 to the PC-side converter model via the onboard Micro-USB (ST-LINK VCP).

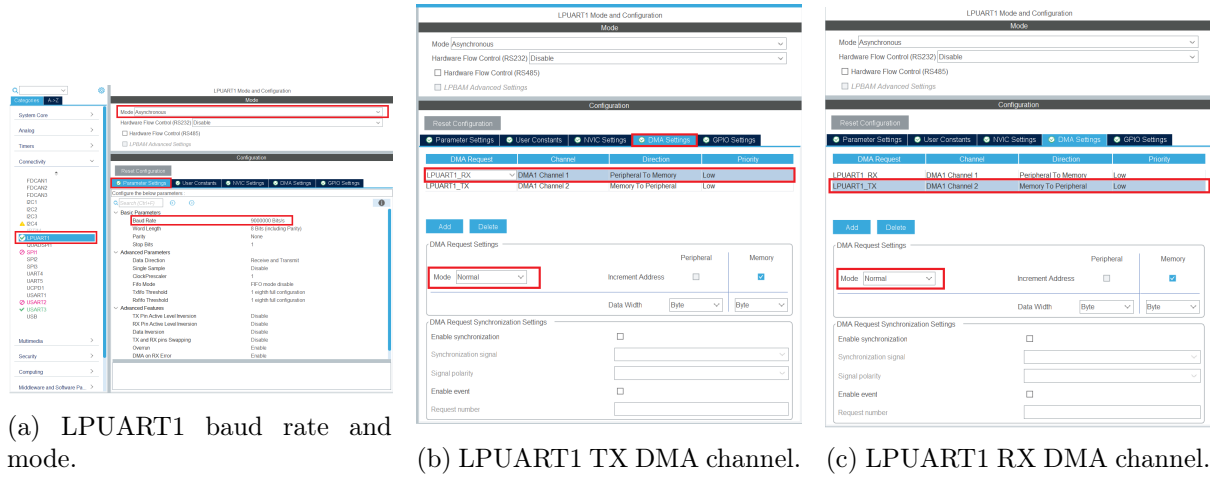


Figure 7: LPUART1 peripheral configuration in CubeMX.

2.3.6 USART3 Configuration

USART3 handles reception of the simulated output voltage from the PC-side converter model, via the external USB-to-USART converter.

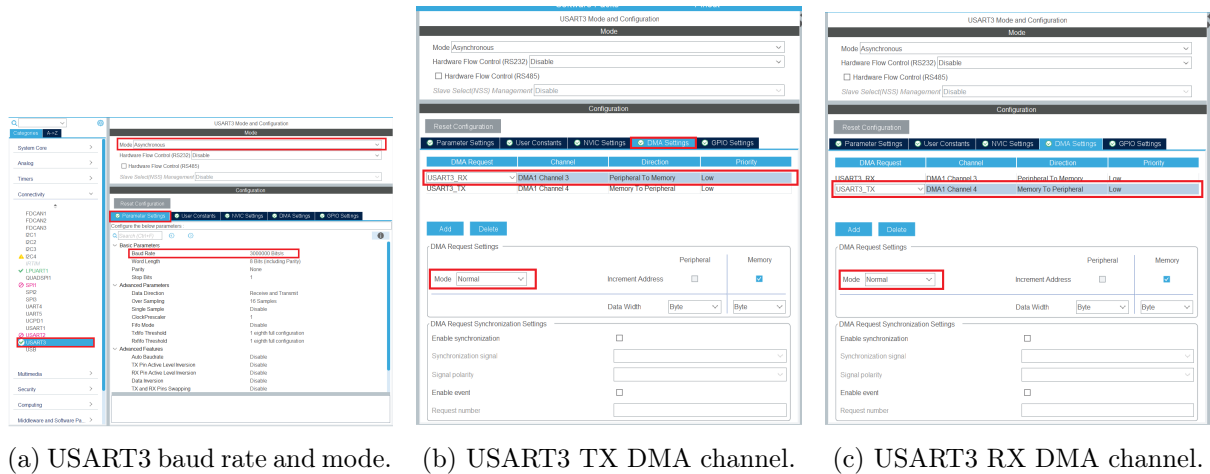


Figure 8: USART3 peripheral configuration in CubeMX.

2.4 Wiring and Hardware Setup

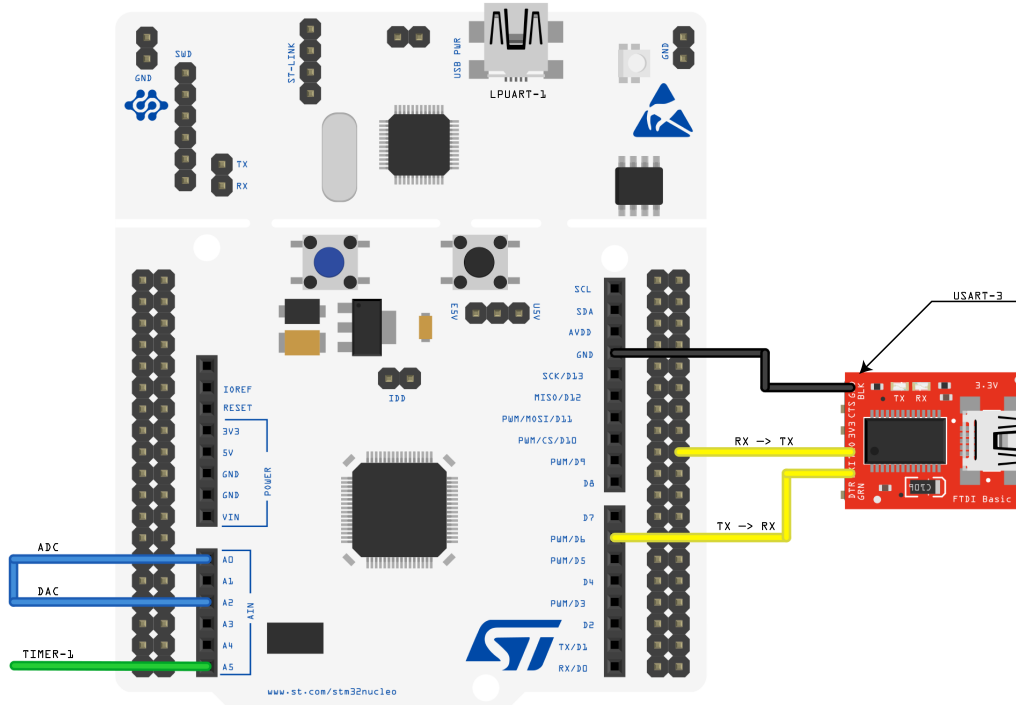


Figure 9: Wiring diagram for the co-simulation setup.

The physical connections required are:

- **Jumper wire:** Short PA0 to PA4 on the NUCLEO board. This closes the analogue feedback loop — the DAC-driven voltage on PA4 is read back by the ADC on PA0.
- **LPUART1 (Micro-USB):** Connect the onboard Micro-USB port to the PC. This carries the duty cycle transmitted by the STM32 to the converter model.
- **USART3 (External converter):** Connect PB10 (TX) and PB11 (RX) to a USB-to-USART converter. The converter carries the simulated output voltage from the PC to the STM32.

Note: Recommended USB-to-USART converters (3.3 V compatible, FTDI chip):

- FTDI FT232RL module - Quartz Component
- Adafruit FT232H breakout
- Sparkfun FT232RL module

Ensure the logic voltage selection jumper (if present) is set to **3.3 V** before connecting to the board.

2.5 Building and Deploying

With both models configured, the deployment process follows the same workflow established in previous projects:

1. Open the controller model in MATLAB/Simulink.
2. Press **CTRL+E** to open Configuration Parameters. Verify the hardware board is set to *STM32G4xx based* and the toolchain is set to *GNU Tools for ARM Embedded Processors*.
3. Connect the NUCLEO board via USB.
4. In the Simulink toolbar, click the **Deploy to Hardware** button (Build, Deploy & Start arrow).
5. Once deployed, launch the converter model on the PC and run it in normal simulation mode.
6. Verify both UART links are active and data is flowing — the co-simulation is now live.

3 Testing and Validation

3.1 Hardware Testing

Before running the full co-simulation, verify each hardware element independently:

1. Confirm the PA0–PA4 jumper is securely in place and both pins are accessible.
2. Verify LPUART1 is visible as a COM port on the PC (Device Manager on Windows).
3. Verify USART3 is visible on the second COM port through the USB-to-USART converter.
4. Check that the USB-to-USART converter logic level is set to 3.3 V.

3.2 Model Simulation Testing

Before deploying to hardware, validate the controller model in Simulink simulation mode:

- Verify the controller converges to V_{ref} under step input conditions.
- Check that the duty cycle remains clamped within 0–95% under all simulated operating conditions.

3.3 Integration Testing

Once both models are running together:

- Confirm duty cycle packets are being received correctly by the converter model (check COM port data).
- Confirm the STM32 DAC is producing a voltage that the ADC reads back within a reasonable tolerance.
- Monitor the output voltage in the converter model's scope — it should settle towards V_{ref} .

4 Results and Analysis

4.1 Experimental Results

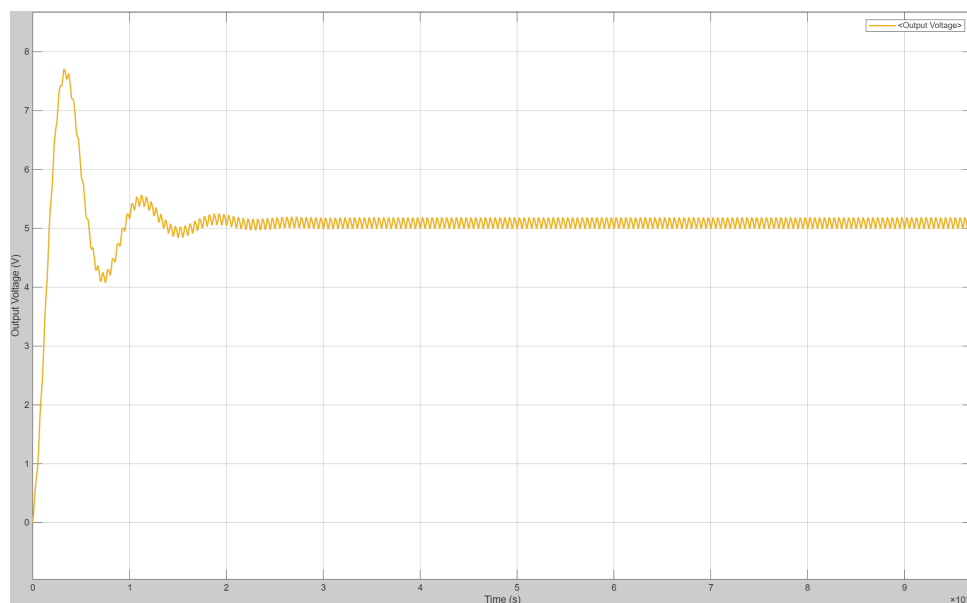


Figure 10: Output voltage settling to approximately 5 V — close-up view.

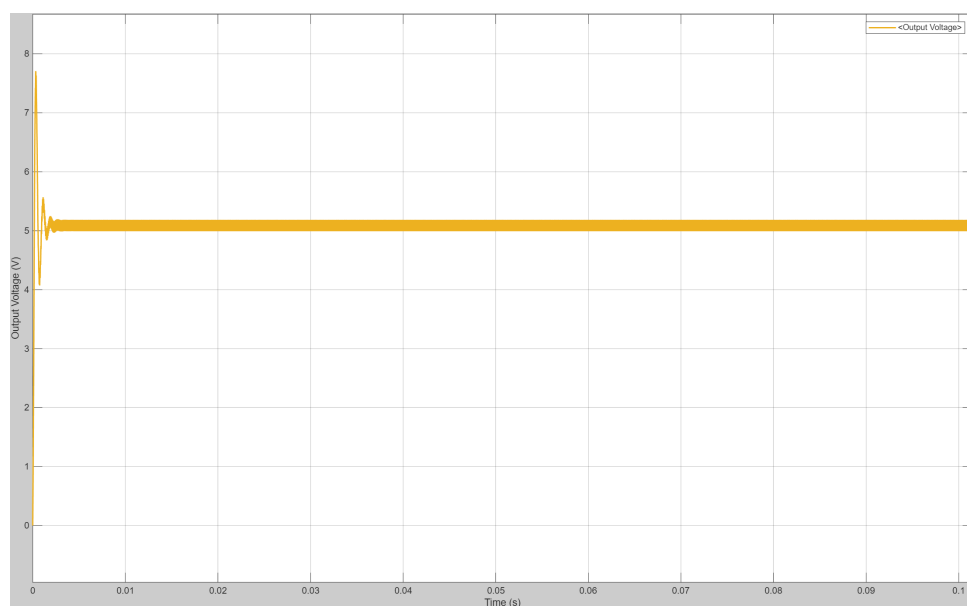


Figure 11: Output voltage — zoomed-out view showing the full transient and steady-state response.

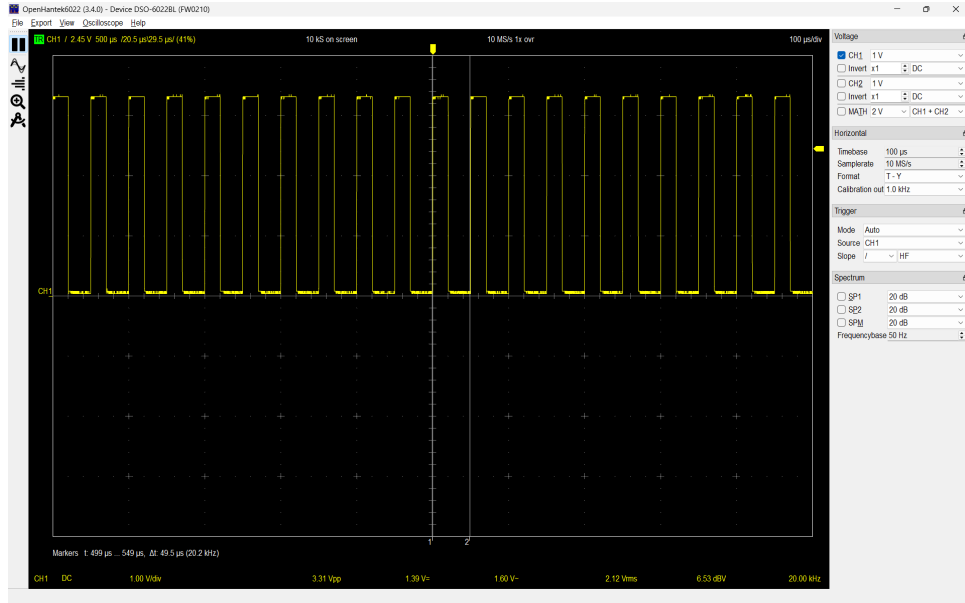


Figure 12: Physical PWM pulses on PA8 (TIM1) once output voltage has settled to V_{ref} .

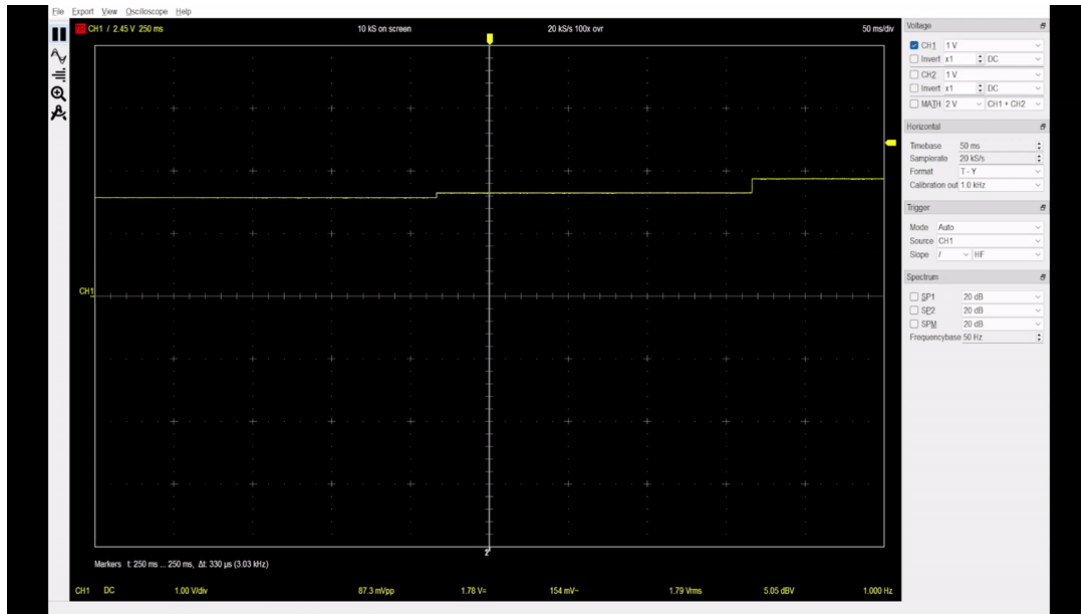


Figure 13: Real-time DAC output voltage illustrating how the converter voltage changes and settles during co-simulation. **Click the image to view the full animation.**

4.2 Key Observations

- The output voltage settles to approximately 5 V with only a marginal steady-state error, demonstrating that the PI controller with feedforward compensation successfully regulates the buck converter.
- The DAC–ADC short arrangement faithfully reconstructs the simulated output voltage on the embedded side, validating the novel analogue feedback approach.
- Physical PWM pulses observed on PA8 confirm the duty cycle computation and TIM1

configuration are functioning correctly.

5 Troubleshooting

Table 2: Common Issues and Solutions

Issue	Solution
Output voltage does not settle	Verify the PA0–PA4 jumper is connected; check ADC and DAC DMA configurations in CubeMX
Duty packets not received by converter model	Confirm the correct COM port is selected in the Simulink serial receive block and baud rates match on both sides
USART3 link silent or corrupted data	Verify USB-to-USART converter is set to 3.3 V logic; check PB10/PB11 TX-RX cross-wiring
Build or deploy fails	Confirm GNU Tools for ARM Embedded Processors is selected as the toolchain in Configuration Parameters

6 Extensions and Enhancements

6.1 Possible Improvements

1. **Broader topology testing:** The current implementation has been validated on the buck converter topology. Testing with boost, buck-boost, and SEPIC topologies would help quantify the feasibility and generalisability of this co-simulation approach.
2. **Control law simplification:** A simplified revision of the `buck_controller` function is scheduled. Reducing the number of conditional branches and persistent states will improve readability and reduce generated code complexity.
3. **Multi-variable co-simulation:** The current architecture is tightly constrained to a single control variable (duty cycle) and a single feedback signal (V_o). Extending the packet protocol to support multiple stream IDs would allow simultaneous transmission of inductor current, input voltage, and other state variables.
4. **Variable reference tracking:** Replace the fixed V_{ref} constant with a dynamic setpoint driven by a step or ramp source in the controller model, enabling closed-loop reference tracking studies.

6.2 Known Limitations

- The system is tightly constrained by UART bandwidth and fixed timing. Adding more control variables may overwhelm the serial links or cause timing violations in the 20 kHz control loop.

- Latency introduced by the serial communication path (PC $\leftrightarrow$ STM32) is not negligible and introduces a non-deterministic delay into the feedback loop. This limits the maximum achievable control bandwidth.

7 Conclusion

This project demonstrates a low-cost, accessible approach to hardware-in-the-loop co-simulation of a closed-loop power converter using the STM32G474RE and MATLAB/Simulink. By splitting the system into a deployed controller and a PC-side Simscape plant, the approach sidesteps the memory constraints that would prevent full on-chip simulation, whilst maintaining a high-fidelity plant model.

The novel use of the DAC–ADC short to physically close the analogue feedback loop distinguishes this approach from conventional UART co-simulations and ensures that the ADC sampling path is exercised as it would be in a real deployment.

Key Takeaways:

- A two-file co-simulation architecture can effectively decouple plant and controller complexity, enabling high-fidelity simulation on resource-constrained hardware.
- Physically closing the loop via the DAC and ADC ensures the embedded sampling chain is validated as part of the co-simulation, rather than being abstracted away.
- DMA-driven peripheral access is essential for sustaining a 20 kHz real-time control loop whilst simultaneously handling multiple UART links.
- This approach, whilst functional, is best suited to single-variable control problems in its current form; broader applicability requires further validation across topologies and operating conditions.

References

- [1] STMicroelectronics, *STM32G4 Series Advanced ARM-based 32-bit MCUs Reference Manual*, RM0440.
- [2] STMicroelectronics, *STM32G474RE Datasheet*.
- [3] STMicroelectronics, *STM32G4 NUCLEO-64 Boards User Manual*, UM2505.
- [4] STMicroelectronics, *STM32CubeIDE Integrated Development Environment User Manual*, DM00629855.
- [5] STMicroelectronics, *Description of STM32G4 HAL and Low-Layer Drivers*, UM2570.
- [6] STMicroelectronics, *HRTIM Cookbook*, AN4539.
- [7] MathWorks, *Embedded Coder Documentation*.
- [8] MathWorks, *Embedded Coder Support Package for STMicroelectronics STM32*.

A Controller: buck_controller Function

The `buck_controller` MATLAB function implements a feedforward PI controller with gain scheduling, asymmetric hysteresis, and anti-windup. It is used inside a MATLAB Function block in the controller Simulink model.

```

1 function D = buck_controller(Vref_in, Vo_in, Vin_in)
2 %#codegen
3 % Feedforward + PI with Sanity Shield, Asymmetric Hysteresis & Anti-
   Windup
4 % Plant: L=450uH, C=30uF, fs=20kHz | UART Co-Simulation Safe
5
6 persistent int_v
7 persistent D_prev
8 persistent Vo_prev
9 persistent is_initialized
10
11 %% ----- Constants -----
12 Kp      = single(0.005);
13 Ki_step = single(1e-5);
14 Dmax    = single(0.95);
15 Dmin    = single(0.00);
16 % Upper deadband: freeze integrator if Vo is within this margin of
   Vref
17 Vband_hi = single(0.30);
18 % Asymmetric lower band: force response if Vo sags more than this
   below Vref
19 % Vband_lo > Vband_hi creates a one-way recovery escape hatch
20 Vband_lo = single(0.60);
21 % Suppress only DOWNWARD D changes (noise rejection); upward
   recoveries bypass
22 D_hyst   = single(0.002);
23
24 %% ----- Initialization -----
25 if isempty(is_initialized)
26     D_prev      = single(Vref_in / Vin_in);
27     int_v       = single(0.0);
28     Vo_prev     = single(0.0);
29     is_initialized = true;
30 end
31
32 %% ----- Type Casting & Sanity Shield -----
33 Vref = single(Vref_in);
34 Vin  = single(Vin_in);
35 Vo_raw = single(Vo_in);
36
37 Vo_max_plausible = Vref + single(5.0); % Hard ceiling
38 Vo_delta_limit   = single(0.20);      % Max 200 mV change per 50

```

```

    us step
39
40 if (Vo_raw > Vo_max_plausible) || (Vo_raw < single(0.0))
41     Vo = Vo_prev; % Hard reject: keep last
    known good
42 elseif abs(Vo_raw - Vo_prev) > Vo_delta_limit
43     Vo = Vo_prev + sign(Vo_raw - Vo_prev) * Vo_delta_limit; % Rate-
    limited nudge
44     Vo_prev = Vo;
45 else
46     Vo = Vo_raw;
47     Vo_prev = Vo;
48 end
49
50 %% ----- Feedforward & Error -----
51 D_ff = Vref / Vin;
52 ev = Vref - Vo;
53
54 % Hard-clamp error magnitude
55 if ev > Vref
56     ev = Vref;
57 elseif ev < -Vref
58     ev = -Vref;
59 end
60
61 %% ----- Band Classification -----
62 in_band = (ev >= single(0.0)) && (ev < Vband_hi); % Freeze
    integrator
63 below_band = ev > Vband_lo; % Emergency
    response
64
65 %% ----- Gain Scheduling -----
66 if in_band
67     Kp_active = single(0.0);
68 else
69     Kp_active = Kp;
70 end
71
72 %% ----- PI Correction -----
73 D_unclamped = D_ff + (Kp_active * ev) + int_v;
74
75 %% ----- Integrator Update (with Anti-Windup) -----
76 if in_band
77     % Gently bleed integrator back toward zero to prevent silent
    accumulation
78     if int_v > single(0.0)
79         int_v = int_v - single(1e-6);
80     elseif int_v < single(0.0)

```

```

81         int_v = int_v + single(1e-6);
82     end
83 elseif D_unclamped > Dmax
84     if ev < single(0.0)
85         int_v = int_v + (ev * Ki_step); % Saturated high: allow only
pull-down
86     end
87 elseif D_unclamped < Dmin
88     if ev > single(0.0)
89         int_v = int_v + (ev * Ki_step); % Saturated low: allow only
push-up
90     end
91 else
92     int_v = int_v + (ev * Ki_step);      % Normal region: accumulate
freely
93 end
94
95 %% ----- Output with Asymmetric Hysteresis -----
96 D_ideal = single(max(Dmin, min(Dmax, D_unclamped)));
97
98 if D_ideal >= D_prev
99     % Upward correction (voltage recovery): bypass hysteresis in
emergencies
100     if below_band || (abs(D_ideal - D_prev) >= D_hyst)
101         D      = D_ideal;
102         D_prev = D_ideal;
103     else
104         D = D_prev;
105     end
106 else
107     % Downward correction: apply hysteresis to suppress noise-driven
decrements
108     if abs(D_ideal - D_prev) >= D_hyst
109         D      = D_ideal;
110         D_prev = D_ideal;
111     else
112         D = D_prev;
113     end
114 end
115 end

```

Listing 1: buck_controller.m — Feedforward PI with Anti-Windup and Asymmetric Hysteresis

B Controller: parse_rx Function

This function is used in the controller model to reconstruct the DAC integer value from the four-byte packet received via USART3.

```

1 function VO_out = parse_rx(buffer, VO_prev)
2 % Reconstructs a uint16 DAC value from a 4-byte packet (header +
   stream ID + MSB + LSB)
3 % Returns VO_prev if no valid packet is found in the buffer
4
5 % Force output type to uint16 for codegen compatibility
6 VO_out = uint16(VO_prev);
7 len = length(buffer);
8
9 for i = 1:(len - 3)
10     if buffer(i) == 170 && buffer(i+1) == 86
11         MSB = uint16(buffer(i+2));
12         LSB = uint16(buffer(i+3));
13         VO_out = bitor(bitshift(MSB, 8), LSB);
14         break;
15     end
16 end
17 end

```

Listing 2: parse_rx.m — Packet parser for incoming voltage data (USART3)

C Converter: parse_uart Function

This function is used in the PC-side converter model to reconstruct the duty cycle from the packet received via LPUART1.

```

1 function D = parse_uart(buffer, status, Dprev)
2 % Reconstructs duty cycle from a 2-byte packet (MSB + LSB)
3 % Returns Dprev if no new data has arrived (status == 0)
4
5 if status == 1
6     % New valid packet: reconstruct 16-bit duty cycle
7     MSB = uint16(buffer(1));
8     LSB = uint16(buffer(2));
9     D = bitor(bitshift(MSB, 8), LSB);
10 else
11     % No new data: hold previous value
12     D = Dprev;
13 end
14 end

```

Listing 3: parse_uart.m — Packet parser for incoming duty cycle data (LPUART1)

D Hardware Documentation

Refer to the official NUCLEO-G474RE schematic and the STM32G474RE Reference Manual (RM0440) for complete hardware specifications, pin definitions, and electrical characteristics.

Author

Anmol Govindarajapuram Krishnan

GitHub: <https://github.com/Anmol-G-K>

Email: cb.en.u4eee23103@cb.students.amrita.edu

Acknowledgments

We acknowledge the following for their contributions and support:

- **STMicroelectronics** for providing comprehensive microcontroller documentation and development tools
- **MathWorks** for excellent MATLAB and Embedded Coder resources
- The embedded systems and power electronics community for innovation and knowledge sharing
- Contributors and reviewers providing feedback

This project is licensed under the MIT License.

For updates and additional resources:

<https://github.com/Anmol-G-K/NUCLEO-G474RE-PowerElectronics-Guide>